Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work work #4.1

Actuator Control and Signal Conversion

During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur Cravtov
std. gr. FAF-231

Verified by:
Alexei Martîniuc
asist. univ

Chișinău, 2026

ableofcontents

1 Objective and Technical Analysis

The objective of this laboratory work is to implement a PWM-based actuator control system with signal conditioning for embedded systems. This involves controlling analog actuators (LED brightness and servo motor position) through Pulse Width Modulation, applying Exponential Moving Average (EMA) filtering for smooth transitions, and providing real-time feedback through display interfaces. The system implements a multi-stage control pipeline: **Input Processing**, **Signal Conditioning (EMA)**, and **PWM Actuator Control with Feedback**.

Through this work, the following learning outcomes are pursued:

- Understanding PWM (Pulse Width Modulation) principles for analog actuator control.
- Implementing EMA (Exponential Moving Average) filtering for signal smoothing.
- Controlling servo motors with precise angular positioning (0-180°).
- Managing real-time input/output through multiple interfaces (keypad, LCD, serial).
- Designing modular software architecture using FreeRTOS with dedicated tasks.
- Implementing abstract interfaces for hardware abstraction and code reusability.

1.1 Pulse Width Modulation (PWM)

1.1.1 PWM Fundamentals

Pulse Width Modulation is a technique used to encode analog signal levels using digital outputs. By varying the duty cycle (the proportion of time the signal is HIGH versus LOW), the average voltage delivered to a load can be controlled [2].

The duty cycle is expressed as a percentage:

$$D = \frac{T_{ON}}{T_{ON} + T_{OFF}} \times 100\% \tag{1}$$

where T_{ON} is the time the signal is HIGH and T_{OFF} is the time the signal is LOW.

The average output voltage is:

$$V_{avg} = D \times V_{CC} \tag{2}$$

1.1.2 PWM for LED Control

For LED brightness control, PWM effectively controls the perceived brightness due to the persistence of vision. The human eye integrates the rapid ON/OFF cycles, perceiving an average brightness level. A duty cycle of 50% results in approximately half brightness [3].

1.1.3 PWM for Servo Motor Control

Servo motors use PWM signals for position control, but with a different encoding than simple duty cycle. Standard hobby servos expect a PWM pulse every 20ms, with pulse widths ranging from 1ms (0°) to 2ms (180°) [4].

The relationship between pulse width and angle:

$$\theta = \frac{t_{pulse} - 1ms}{1ms} \times 180 \tag{3}$$

The Arduino Servo library abstracts this, allowing direct angle specification (0-180°).

1.2 Exponential Moving Average (EMA) Filtering

1.2.1 EMA Algorithm

EMA is a type of infinite impulse response (IIR) filter that applies weighting factors which decrease exponentially. It is used to smooth signals and reduce noise while maintaining responsiveness to genuine changes [5].

The EMA formula is:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n - 1] \tag{4}$$

where:

- $y[n]$ is the filtered output at time n
- $x[n]$ is the raw input at time n
- α is the smoothing factor (0 ¡ α ¡ 1)

1.2.2 EMA Characteristics

The smoothing factor α determines the filter's behavior:

- Lower α (e.g., 0.1): More smoothing, slower response, better noise rejection
- Higher α (e.g., 0.5): Less smoothing, faster response, more noise passes through

The effective time constant of the EMA filter:

$$\tau = \frac{1-\alpha}{\alpha} \text{ samples} \tag{5}$$

For $\alpha = 0.1$, the time constant is 9 samples. The filter reaches approximately 63% of a step change in one time constant and 99% in approximately 5 time constants (45 samples).

1.2.3 Advantages of EMA

- Computationally efficient: requires only one multiplication, one subtraction, and one addition per sample
- Memory efficient: only one previous output value needs to be stored
- Suitable for embedded systems with limited resources
- No buffer required (unlike median or moving average filters)

1.3 Hardware Abstraction Layer

Hardware abstraction provides a clean separation between application logic and low-level hardware access. This design pattern enables code portability and simplifies testing [6].

The abstract base class pattern defines the actuator interface:

- `setDutyCycle(uint8_t duty)`: Sets the PWM duty cycle (0-100%)
- `getDutyCycle()`: Returns the current duty cycle
- `begin()`: Initializes the hardware

2 System Design

2.1 System Architecture

The system architecture follows a modular task-based design using FreeRTOS. The application is divided into independent tasks, each with specific responsibilities:

- **TaskRead**: Reads user input from STDIO (keypad or serial) and updates the raw duty cycle. Runs at 60ms intervals.
- **TaskConditioning**: Applies EMA filtering to the raw duty cycle. Runs at 50ms intervals.

- **TaskRamp**: Implements auto-ramp mode for continuous sweeping. Runs at 100ms intervals.
- **TaskWrite**: Updates actuators and LCD display. Runs at 200ms intervals.

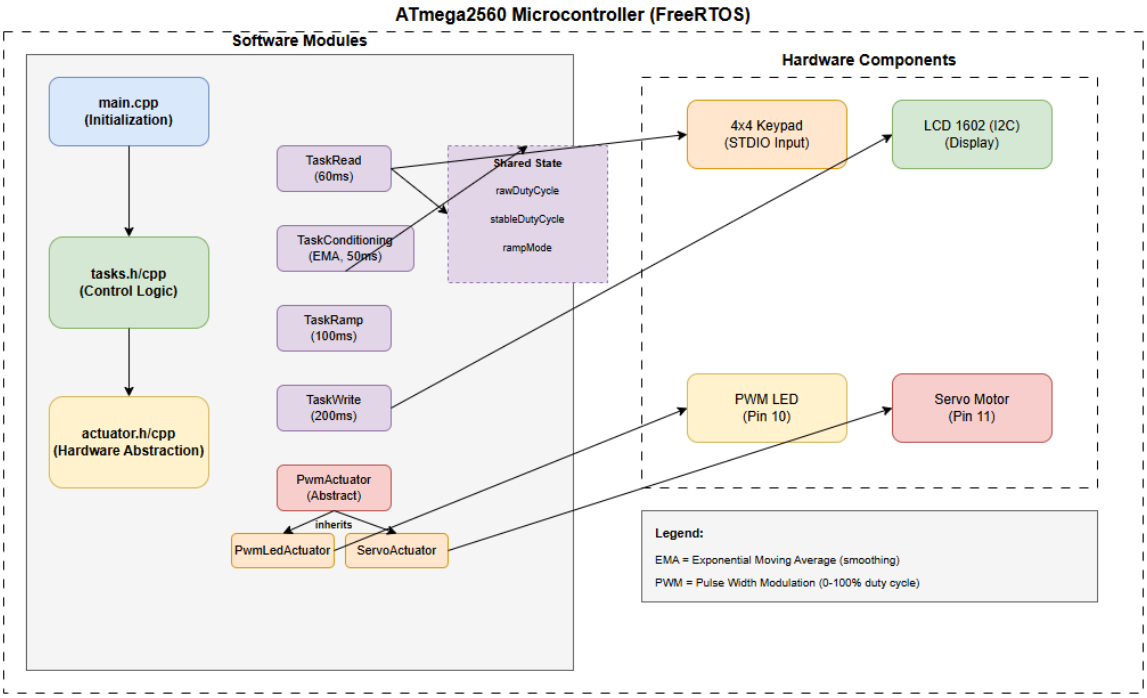

Figure 1: System Architecture Diagram for Lab 4.2

The architecture demonstrates separation of concerns:

- Input handling is decoupled from signal conditioning
- Signal conditioning is isolated in a dedicated task
- Output operations are grouped for efficient timing
- Shared state enables inter-task communication

2.2 Circuit Diagram

Figure 2 shows the hardware connections between the ATmega2560 microcontroller and the external components. The circuit implements a PWM actuator control system with LED and servo motor outputs.

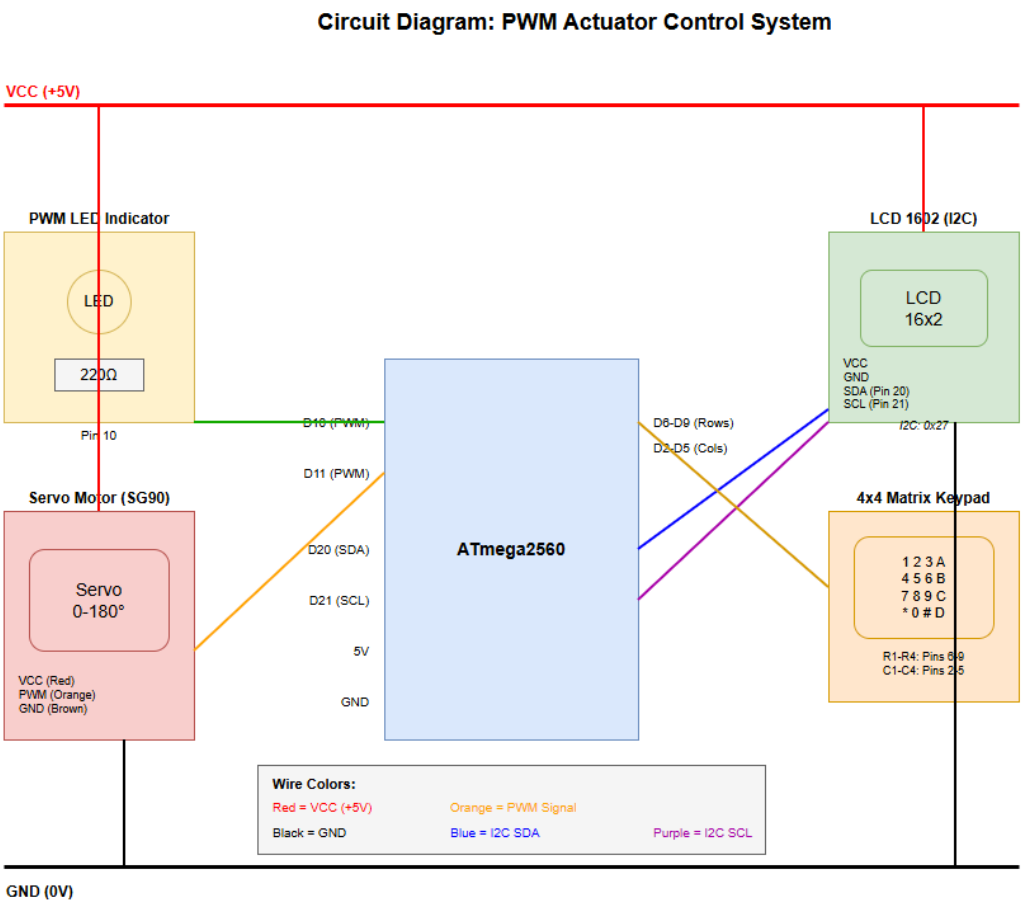

Figure 2: Circuit Diagram: PWM Actuator Control System

2.2.1 PWM LED Connections

The LED provides visual feedback of the duty cycle level through brightness variation.

- **Anode (+)**: Connected to digital pin D10 via a 220Ω current-limiting resistor
- **Cathode (-)**: Connected to ground
- **Purpose**: Visual indication of PWM duty cycle (brightness = duty cycle)

2.2.2 Servo Motor (SG90) Connections

The servo motor provides angular position control based on duty cycle.

- **VCC (Red)**: Connected to 5V power supply. The SG90 requires 4.8-6V and draws approximately 100-250mA during operation.
- **GND (Brown)**: Connected to ground rail
- **Signal (Orange)**: Connected to digital pin D11. Receives PWM control signal.
- **Position Mapping**: 0% duty = 0°, 100% duty = 180°

2.2.3 LCD 1602 (I2C) Connections

The LCD display provides real-time feedback about the system state.

- **VCC**: Connected to 5V
- **GND**: Connected to ground
- **SDA**: Connected to pin D20 (I2C data line)
- **SCL**: Connected to pin D21 (I2C clock line)
- **I2C Address**: 0x27

2.2.4 4x4 Keypad Connections

The matrix keypad provides user input for duty cycle control.

- **Row Pins (R1-R4)**: Connected to D9, D8, D7, D6
- **Column Pins (C1-C4)**: Connected to D5, D4, D3, D2

2.2.5 Software Layer Architecture

Figure 3 illustrates the layered software architecture for servo motor control, following the AUTOSAR-style layer model.

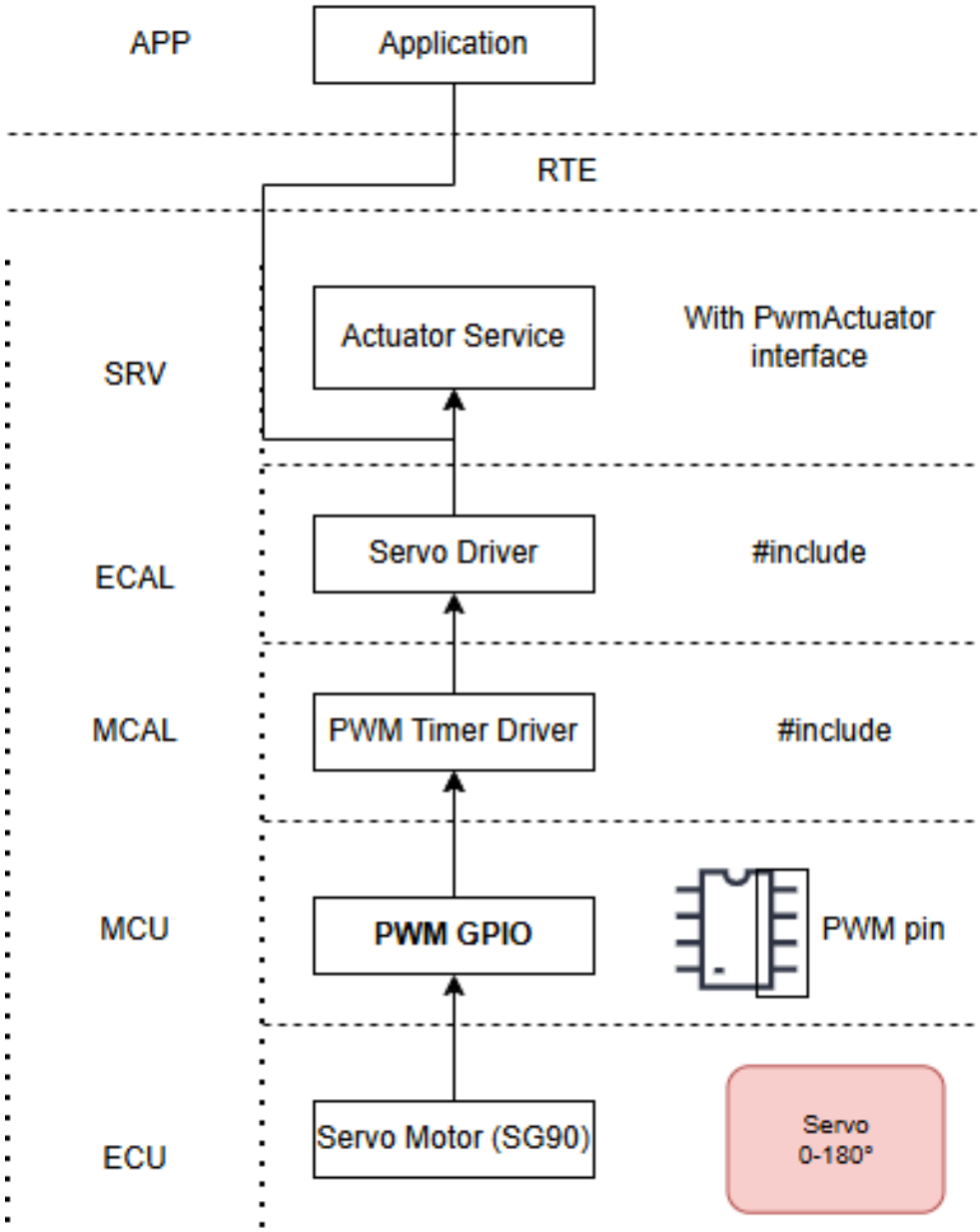

Figure 3: Servo Actuator Software Layers

The layers from bottom to top:

- **ECU (Electronic Control Unit)**: The physical servo motor hardware
- **MCU (Microcontroller Unit)**: PWM-capable GPIO pins
- **MCAL (Microcontroller Abstraction Layer)**: PWM Timer Driver, low-level hardware access
- **ECAL (ECU Abstraction Layer)**: Servo Driver, abstracts servo-specific PWM timing
- **SRV (Service Layer)**: Actuator Service with PwmActuator interface

- **APP (Application Layer)**: User application logic

2.2.6 Pin Assignment Summary

Table 1 summarizes the complete pin assignments.

Table 1: Microcontroller Pin Assignments

Component	**Pin**	**Function**
PWM LED	D10	PWM output (Timer4)
Servo Signal	D11	PWM output (Timer4)
LCD SDA	D20	I2C data
LCD SCL	D21	I2C clock
Keypad R1	D9	Row 1 (output)
Keypad R2	D8	Row 2 (output)
Keypad R3	D7	Row 3 (output)
Keypad R4	D6	Row 4 (output)
Keypad C1	D5	Column 1 (input)
Keypad C2	D4	Column 2 (input)
Keypad C3	D3	Column 3 (input)
Keypad C4	D2	Column 4 (input)

2.3 EMA Conditioning Pipeline

Figure 4 illustrates the signal conditioning flow through the EMA filter.

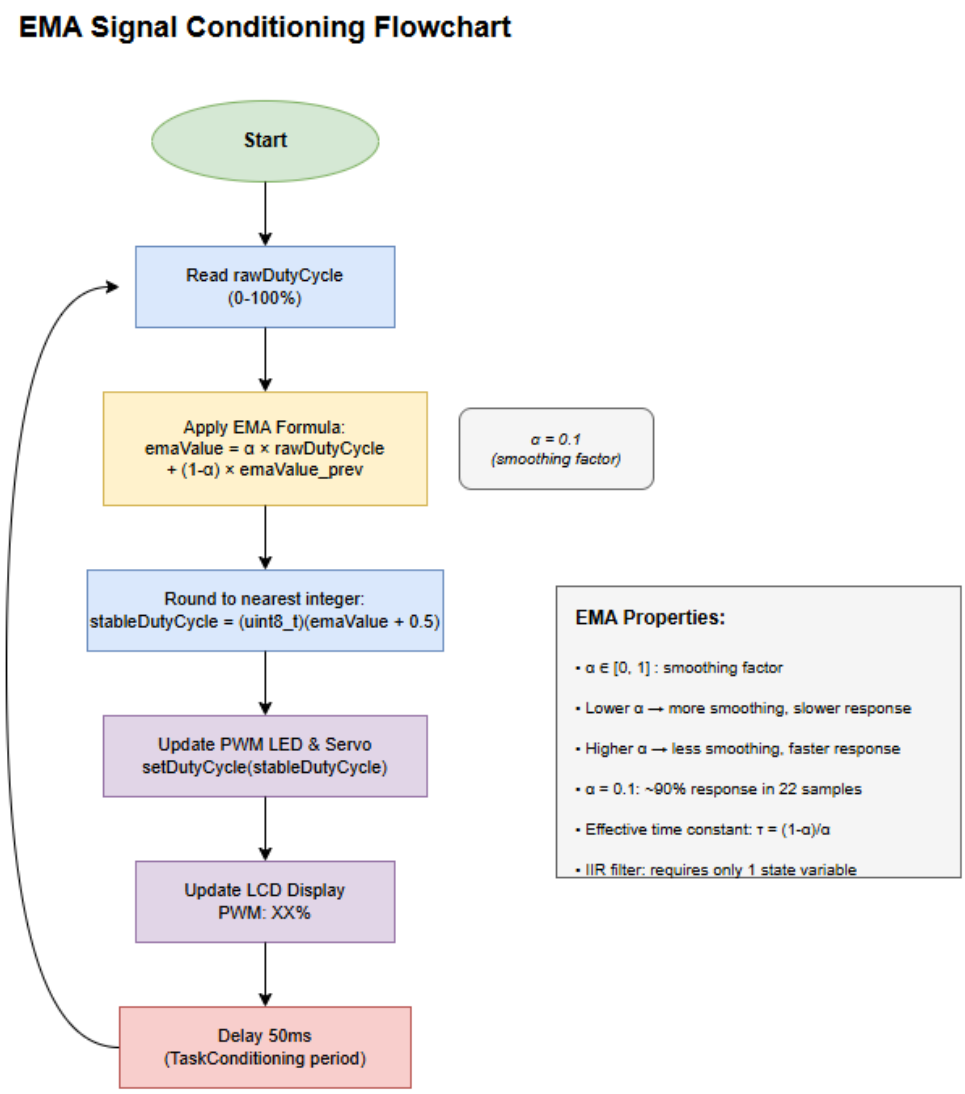

Figure 4: EMA Signal Conditioning Flowchart

The EMA conditioning process:

1. Read the raw duty cycle from shared state
2. Apply the EMA formula with $\alpha = 0.1$
3. Round the result to the nearest integer
4. Store in stableDutyCycle for actuator tasks
5. Delay 50ms before next iteration

This approach ensures:

- Smooth transitions between duty cycle levels
- No abrupt changes that could cause mechanical stress on the servo

- Pleasant visual fading effect on the LED
- Noise filtering from user input

3 Implementation and Results

3.1 Software Architecture

The implementation follows a modular object-oriented approach within the C++ framework provided by Arduino.

3.1.1 Module Organization

- **actuator.h / actuator.cpp**: Contains the abstract `PwmActuator` interface and concrete implementations (`PwmLedActuator`, `ServoActuator`).
- **tasks.h / tasks.cpp**: Contains all FreeRTOS task implementations and shared state variables.
- **main.cpp**: Contains system initialization and task creation.

3.1.2 PwmActuator Abstract Interface

```
class PwmActuator {
  public:
    virtual void setDutyCycle(uint8_t duty) = 0;
    virtual uint8_t getDutyCycle() = 0;
    virtual void begin() = 0;
    virtual ~PwmActuator() {}
};
```

3.1.3 ServoActuator Implementation

```
class ServoActuator : public PwmActuator {
  private:
    uint8_t pin;
    uint8_t dutyCycle;
    bool initialized;
  public:
    ServoActuator(uint8_t p);
    void begin() override;
    void setDutyCycle(uint8_t duty) override {
        if (duty > 100) duty = 100;
        dutyCycle = duty;
        int angle = map(dutyCycle, 0, 100, 0, 180);
        servo.write(angle);
    }
    uint8_t getDutyCycle() override { return dutyCycle; }
};
```

3.1.4 TaskConditioning: EMA Implementation

```c
void TaskConditioning(void* pvParameters) {
    const float alpha = 0.1f;  // Smoothing factor
    float emaValue = 0.0f;

    while(true) {
        // EMA formula: y[n] = alpha * x[n] + (1-alpha) * y[n-1]
        emaValue = alpha * (float)rawDutyCycle
                 + (1.0f - alpha) * emaValue;

        // Round to nearest integer
        stableDutyCycle = (uint8_t)(emaValue + 0.5f);

        vTaskDelay(pdMS_TO_TICKS(50));
    }
}
```

3.1.5 TaskRead: Input Processing

```c
void TaskRead(void* pvParameters) {
    while(true) {
        char c = 0;
        int result = scanf("%c", &c);

        if (result > 0 && c != '\0' && c != '\n' && c != '\r') {
            if (c >= '0' && c <= '9') {
                rawDutyCycle = (c - '0') * 10;  // 0-90%
                rampMode = false;
            } else if (c == 'A' || c == 'a') {
                rawDutyCycle = 100;  // 100%
                rampMode = false;
            } else if (c == '*') {
                rampMode = !rampMode;  // Toggle ramp
            } else if (c == '#') {
                rawDutyCycle = 0;  // Reset
                rampMode = false;
            }
            // ... additional key handling
        }
        vTaskDelay(pdMS_TO_TICKS(60));
    }
}
```

3.1.6 TaskWrite: Actuator Update

```c
void TaskWrite(void* pvParameters) {
    uint8_t lastDuty = 255;  // Force initial update

    while(true) {
        uint8_t currentDuty = stableDutyCycle;

        if (currentDuty != lastDuty) {
            if (g_pwmLed) g_pwmLed->setDutyCycle(currentDuty);
            if (g_servo) g_servo->setDutyCycle(currentDuty);
            lastDuty = currentDuty;
        }

        // Update LCD display
        printf("\r                    \r");
        printf("PWM: %3d%%", currentDuty);
        if (rampMode) printf(" [RAMP]");
        printf("\r");

        vTaskDelay(pdMS_TO_TICKS(200));
    }
}
```

3.2 Task Synchronization

The system uses volatile variables for inter-task communication:

```
volatile uint8_t rawDutyCycle = 30;    // Start at 30%
volatile uint8_t stableDutyCycle = 0;
volatile bool rampMode = false;
```

The `volatile` keyword ensures that the compiler does not optimize away reads/writes to these variables. Given the simple data types and single-writer pattern, explicit mutex protection is not required.

3.3 Keypad Control Mapping

Table 2 shows the keypad key assignments for duty cycle control.

Table 2: Keypad Key Assignments

Key	**Function**
0-9	Set duty cycle to 0%, 10%, 20%, ..., 90%
A	Set duty cycle to 100%
B	Increment duty cycle by 10%
C	Decrement duty cycle by 10%
D	Set duty cycle to 50% (center)
*	Toggle auto-ramp mode
#	Reset to 0%

3.4 Testing and Validation

Scenario 1: Manual Control

- Press '5' → LED brightness to 50%, servo to 90°
- Press 'A' → LED full brightness, servo to 180°
- Press '#' → LED off, servo to 0°
- EMA smoothing creates gradual transitions (1-2 seconds)

Scenario 2: Auto-Ramp Mode

- Press '*' → System enters ramp mode
- Duty cycle automatically sweeps 0% → 100% → 0%
- LED fades up and down smoothly

- Servo rotates back and forth continuously
- Full sweep cycle: approximately 5 seconds

Scenario 3: Increment/Decrement

- Press 'B' → Increase duty cycle by 10%
- Press 'C' → Decrease duty cycle by 10%
- Each step applies EMA smoothing

3.5 Performance Characteristics

Table 3: System Performance Metrics

Metric	**Value**
TaskRead period	60ms
TaskConditioning period	50ms
TaskRamp period	100ms
TaskWrite period	200ms
EMA smoothing factor (α)	0.1
EMA time constant	9 samples (450ms)
Response time (99%)	2.25 seconds
Servo range	0-180°
PWM frequency	490Hz (Arduino default)

3.6 Simulation Results

The system was validated using Wokwi simulation platform. The simulation successfully demonstrated all functionality including PWM actuator control, EMA smoothing, and auto-ramp mode.

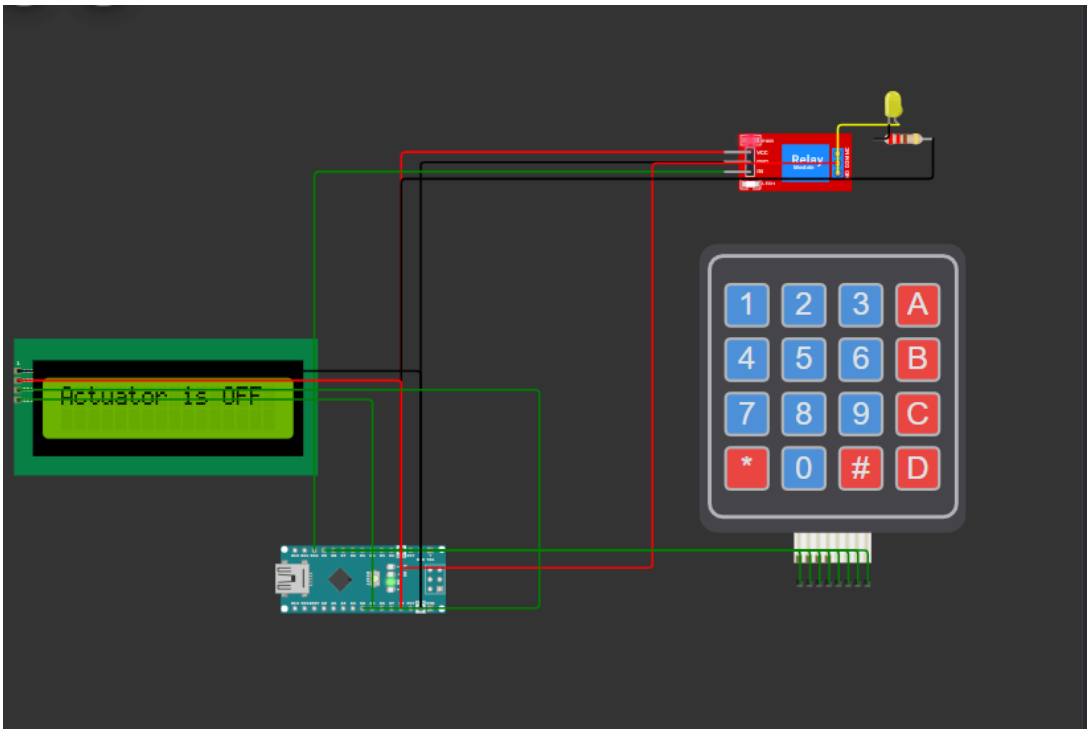

Figure 5: Wokwi Simulation - System at 30% Duty Cycle

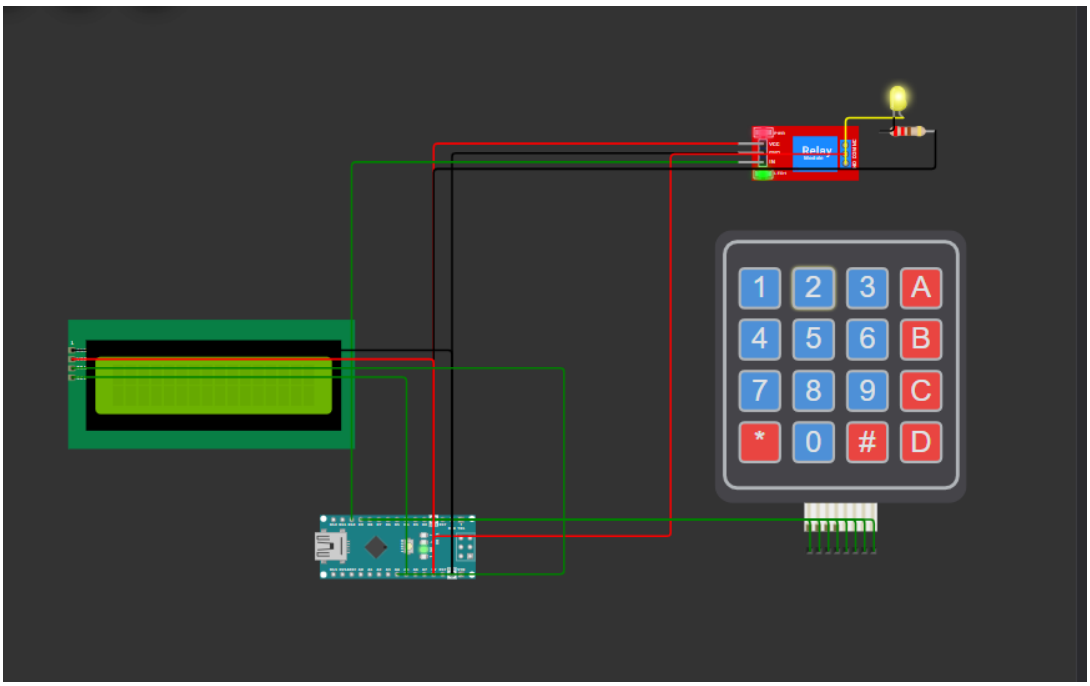

Figure 6: Wokwi Simulation - System at 100% Duty Cycle

4 Conclusion

This laboratory work successfully implemented a PWM-based actuator control system with EMA signal conditioning. The modular architecture using FreeRTOS provided

clear separation of concerns, with dedicated tasks for input processing, signal conditioning, and actuator control.

Key achievements include:

- Smooth PWM actuator control with EMA filtering
- Dual actuator support (LED brightness and servo position)
- Auto-ramp mode for continuous sweeping
- Multi-modal user input (keypad and serial)
- Real-time feedback through LCD and serial interfaces
- Clean software architecture with hardware abstraction

The EMA filtering proved effective for creating smooth transitions, preventing abrupt changes that could cause mechanical stress on the servo motor. The $\alpha = 0.1$ parameter provided a good balance between responsiveness and smoothing.

Future enhancements could include:

- Configurable EMA parameters via user interface
- Multiple servo support with independent control
- Position feedback from servo potentiometer
- PID control for precise positioning
- Speed control mode (rate of change limiting)

References

[1] FreeRTOS API Reference, `https://www.freertos.org/a00106.html`

[2] Barr, M., "Introduction to Pulse Width Modulation", Embedded Systems Programming, 2001.

[3] Winder, S., "LED Power Control Using PWM", Power Electronics Design, 2011.

[4] Hobby Servo Fundamentals, Princeton University, `https://www.princeton.edu/~mae412/HANDOUTS/Servos.pdf`

[5] Smith, S.W., "The Scientist and Engineer's Guide to Digital Signal Processing", California Technical Publishing, 1997.

[6] Noergaard, T., "Embedded Systems Architecture: A Comprehensive Guide for Engineers and Programmers", 2nd ed., Newnes, 2012.

[7] Barr, M., Massa, A., "Programming Embedded Systems in C and C++", O'Reilly Media, 2006.

[8] Arduino Servo Library Reference, `https://www.arduino.cc/reference/en/libraries/servo/`

A Source Code

Full source code for Lab 4.2 is available in the repository.

`actuator.h`

```
#pragma once

#include <Arduino.h>

// Actuator state structure
typedef struct {
  uint8_t raw;           // Raw input (0-100%)
  uint8_t saturated;     // After saturation limits
  uint8_t filtered;      // After median filter
  uint8_t smoothed;      // After weighted average
  uint8_t output;        // Final output (after ramping)
  bool limit_alert;      // Limit reached alert
} ActuatorState;

// Actuator interface
class AnalogActuator {
  public:
    virtual void setRaw(uint8_t value) = 0;
    virtual void update() = 0;  // Process conditioning pipeline
    virtual uint8_t getSpeed() = 0;
    virtual ActuatorState getState() = 0;
    virtual void begin() = 0;
    virtual ~AnalogActuator() {}
};

// PWM LED actuator
```

```
class PwmLed : public AnalogActuator {
  private:
    uint8_t pin;
    ActuatorState state;
    uint8_t median_buf[3];
    uint8_t median_idx;
    float ema_value;
    uint8_t target;
    uint8_t current;

  public:
    PwmLed(uint8_t p);
    void begin() override;
    void setRaw(uint8_t value) override;
    void update() override;
    uint8_t getSpeed() override { return state.output; }
    ActuatorState getState() override { return state; }
};

// Servo actuator (0-100% => 0-180)
class ServoMotor : public AnalogActuator {
  private:
    uint8_t pin;
    ActuatorState state;
    uint8_t median_buf[3];
    uint8_t median_idx;
    float ema_value;
    uint8_t target;
    uint8_t current;
    Servo servo;

  public:
    ServoMotor(uint8_t p);
    void begin() override;
    void setRaw(uint8_t value) override;
    void update() override;
    uint8_t getSpeed() override { return state.output; }
    ActuatorState getState() override { return state; }
```

```
};
```

Listing 1: actuator.h - Actuator Interface

actuator.cpp

```
#include "actuator.h"
#include <Servo.h>

// Helper: median of 3 values
static uint8_t median3(uint8_t a, uint8_t b, uint8_t c) {
  if (a > b) { uint8_t t = a; a = b; b = t; }
  if (b > c) { uint8_t t = b; b = c; c = t; }
  if (a > b) return a;
  return b;
}

// Helper: saturate to limits
static uint8_t saturate(uint8_t val, uint8_t min_val, uint8_t max_val) {
  if (val < min_val) return min_val;
  if (val > max_val) return max_val;
  return val;
}

// PwmLed implementation
PwmLed::PwmLed(uint8_t p) : pin(p), ema_value(0), target(0), current(0) {
  state = {0, 0, 0, 0, 0, false};
  median_buf[0] = median_buf[1] = median_buf[2] = 0;
  median_idx = 0;
}

void PwmLed::begin() {
  pinMode(pin, OUTPUT);
  analogWrite(pin, 0);
}

void PwmLed::setRaw(uint8_t value) {
  state.raw = value;
}
```

```

void PwmLed::update() {
  // 1. Saturation (0-100%)
  state.saturated = saturate(state.raw, 0, 100);
  state.limit_alert = (state.raw > 100);

  // 2. Median filter (remove impulse noise)
  median_buf[median_idx] = state.saturated;
  median_idx = (median_idx + 1) % 3;
  state.filtered = median3(median_buf[0], median_buf[1],
     median_buf[2]);

  // 3. Weighted average / EMA (reduce fluctuations)
  ema_value = 0.2f * state.filtered + 0.8f * ema_value;
  state.smoothed = (uint8_t)(ema_value + 0.5f);

  // 4. Ramping (soft start/stop)
  target = state.smoothed;
  if (current < target) current++;
  else if (current > target) current--;
  state.output = current;

  // Apply output
  analogWrite(pin, map(state.output, 0, 100, 0, 255));
}

// ServoMotor implementation
static Servo servo;

ServoMotor::ServoMotor(uint8_t p) : pin(p), ema_value(0), target
   (0), current(0) {
  state = {0, 0, 0, 0, 0, false};
  median_buf[0] = median_buf[1] = median_buf[2] = 0;
  median_idx = 0;
}

void ServoMotor::begin() {
  servo.attach(pin);
```

```
  servo.write(0);
}

void ServoMotor::setRaw(uint8_t value) {
  state.raw = value;
}

void ServoMotor::update() {
  // 1. Saturation
  state.saturated = saturate(state.raw, 0, 100);
  state.limit_alert = (state.raw > 100);

  // 2. Median filter
  median_buf[median_idx] = state.saturated;
  median_idx = (median_idx + 1) % 3;
  state.filtered = median3(median_buf[0], median_buf[1],
      median_buf[2]);

  // 3. Weighted average / EMA
  ema_value = 0.2f * state.filtered + 0.8f * ema_value;
  state.smoothed = (uint8_t)(ema_value + 0.5f);

  // 4. Ramping
  target = state.smoothed;
  if (current < target) current++;
  else if (current > target) current--;
  state.output = current;

  // Apply to servo (0-100% => 0-180)
  servo.write(map(state.output, 0, 100, 0, 180));
}
```

Listing 2: actuator.cpp - Actuator Implementation

`tasks.h`

```
#ifndef TASKS_H
#define TASKS_H

#include "actuator.h"
#include <Arduino_FreeRTOS.h>
```

```

// Shared state variables (accessed by tasks)
extern volatile uint8_t rawDutyCycle;
extern volatile uint8_t stableDutyCycle;
extern volatile bool rampMode;

// Global actuator pointers (set in main.cpp)
extern PwmLed* g_pwmLed;
extern ServoMotor* g_servo;

// Task function declarations
void TaskRead(void *pvParameters);
void TaskConditioning(void *pvParameters);
void TaskWrite(void *pvParameters);
void TaskRamp(void *pvParameters);

#endif // TASKS_H
```

Listing 3: tasks.h - Task Interface

tasks.cpp

```
#include "tasks.h"
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <serialio/serialio.h>
#include <lcd/LcdStdioManager.h>

// Shared state
volatile uint8_t rawDutyCycle = 30;
volatile uint8_t stableDutyCycle = 30;
volatile bool rampMode = false;

// Global actuator pointers
PwmLed* g_pwmLed = nullptr;
ServoMotor* g_servo = nullptr;

// TaskRead: keypad input
void TaskRead(void *pvParameters) {
  while (true) {
    char c = 0;
```

```
    int result = scanf("%c", &c);

    if (result > 0 && c != '\0' && c != '\n' && c != '\r') {
      if (c >= '0' && c <= '9') {
        rawDutyCycle = (c - '0') * 10;
      } else if (c == 'A' || c == 'a') {
        rawDutyCycle = 100;
      } else if (c == 'B' || c == 'b') {
        if (rawDutyCycle <= 90) rawDutyCycle += 10;
      } else if (c == 'C' || c == 'c') {
        if (rawDutyCycle >= 10) rawDutyCycle -= 10;
      } else if (c == 'D' || c == 'd') {
        rawDutyCycle = 50;
      } else if (c == '#') {
        rawDutyCycle = 0;
      } else if (c == '*') {
        rampMode = !rampMode;
      }
    }
    vTaskDelay(pdMS_TO_TICKS(60));
  }
}

// TaskConditioning: update actuators
void TaskConditioning(void *pvParameters) {
  while (true) {
    if (g_pwmLed) g_pwmLed->setRaw(rawDutyCycle);
    if (g_servo) g_servo->setRaw(rawDutyCycle);

    // Update conditioning pipeline
    if (g_pwmLed) g_pwmLed->update();
    if (g_servo) g_servo->update();

    vTaskDelay(pdMS_TO_TICKS(50));
  }
}

// TaskRamp: handle ramp mode display
```

```
void TaskRamp(void *pvParameters) {
  while (true) {
    fprintf(stderr, "Ramp mode: %s\n", rampMode ? "ON" : "OFF");
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}

// TaskWrite: display output
void TaskWrite(void *pvParameters) {
  while (true) {
    ActuatorState ledState = g_pwmLed ? g_pwmLed->getState()
                                      : ActuatorState{0, 0, 0,
                                        0, 0, false};
    ActuatorState servoState = g_servo ? g_servo->getState()
                                       : ActuatorState{0, 0, 0,
                                         0, 0, false};

    // Clear screen and display
    printf("\x1B[2J\x1B[H");
    printf("LED:%3d%% Sv:%3d%%\n", ledState.output, servoState.
       output);
    printf("Ramp: %s\n", rampMode ? "ON" : "OFF");

    // Debug output
    if (ledState.limit_alert || servoState.limit_alert) {
      fprintf(stderr, "WARNING: Value exceeded 100%%!\n");
    }

    vTaskDelay(pdMS_TO_TICKS(200));
  }
}
```

Listing 4: tasks.cpp - Task Implementation

main.cpp

```
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <Keypad.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
```

```
#include <lcd/LcdStdioManager.h>
#include <keypad/KeypadStdioManager.h>
#include "actuator.h"
#include <serialio/serialio.h>
#include "tasks.h"

// LCD and Keypad configuration
LiquidCrystal_I2C lcd(0x27, 16, 2);

byte rowPins[4] = {9, 8, 7, 6};
byte colPins[4] = {5, 4, 3, 2};

char keys[4][4] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, 4, 4)
    ;

// Global actuator instances
// PWM LED on pin 10
PwmLed pwmLed(10);
// Servo on pin 11
ServoMotor servo(11);

void setup() {
  Serial.begin(9600);
  pwmLed.begin();
  servo.begin();

  // Set global actuator pointers for tasks
  g_pwmLed = &pwmLed;
  g_servo = &servo;

  lcd.init();
```

```
  lcd.backlight();

  static LcdStdioManager lcdManager;
  lcdManager.setup(&lcd);
  KeypadStdioManager::setup(&keypad);

  redirectSerialToStdio(false, false, true);

  // Create FreeRTOS tasks
  xTaskCreate(TaskRead, "ReadTask", 200, NULL, 1, NULL);
  xTaskCreate(TaskConditioning, "CondTask", 200, NULL, 1, NULL);
  xTaskCreate(TaskRamp, "RampTask", 200, NULL, 1, NULL);
  xTaskCreate(TaskWrite, "WriteTask", 200, NULL, 1, NULL);

  fprintf(stderr, "Servo Motor Control System\n");
  fprintf(stderr, "Keys: 0-9=0-90%%, A=100%%, B=+10%%, C=-10%%,
     D=50%%\n");
  fprintf(stderr, "* = Ramp mode, # = Reset\n");
  fprintf(stderr, "Setup complete. Starting scheduler...\n");
  vTaskStartScheduler();
}

void loop() {}
```

Listing 5: main.cpp - System Initialization